Higher Technological Institute (H.T.I.)
Department of Electronics & Electrical Communications Engineering

Traffic Control With Traffic Announcement

Submitted by :

Zaid Abd Elmonem Adly	**20200116**
Andro Khalid Refaat	**20200186**
Ahmed Ossama Abdel Qader	**20210499**
Ahmed Samir Ali Hassan	**20210506**

Submitted To :
Prof. Mohamed Torad
Eng : Ahmed Ali

Course Code : EEC 214
Group : 4

Table of Contents

1. Objective 3
2. Equipment & Components 3
3. Theory 5
3.1 Signal Timing 5
3.2 Seven-Segment Display 5
3.3 Green Blink Warning Phase 6
3.4 Parallel LCD Interface 6
3.5 DHT11 Temperature & Humidity Sensor 6
4. Procedure 7
5. Circuit Diagram 7
6. Code Explanation 8
Section 1 — Macros & Configurations 8
1. Clock & Include Directives 8
2. Bit-Manipulation Macros 8
3. GPIO Abstraction Macros 9
Section 2 — Parallel LCD Display 9
4. LCD Pin Definitions 9
5. lcd_send_command() 10
6. lcd_init() 10
7. lcd_send_data() 11
8. lcd_write_string() & lcd_set_cursor() 11
Section 3 — DHT11 Sensor 12
9. DHT_Setup() 12
10. DHT_Request() 12
11. DHT_Receive_Data() 12
12. DHT_Read() 13
Section 4 — Main Application & Logic 13
13. LED Pin Definitions 13
14. update_lcd() 13
15. main() — Initialisation 14
16. main() — Cycle 1: Road 1 Green 14
17. main() — Cycle 2: Road 2 Green 15
7. Results 16
8. Conclusion 17
9. References 17

Table of Figures & Tables

Figure 1 — Proteus Circuit Simulation Diagram 7
Table 1 — Equipment & Components 3
Table 2 — Signal Timing Phases 5
Table 3 — Green Blink Warning Phase Behaviour 6
Table 4 — LCD Safety Message vs. Remaining Time 6

Table 5 — Test Results Summary 16

1. Objective

The primary objective of this laboratory project is to design and implement a fully functional four-way traffic light control system using an ATmega32 microcontroller running at 8 MHz. The system simulates a real-world signalised intersection with the following specific goals:

- Control four traffic signals organised into two opposing groups, ensuring that conflicting movements are never simultaneously permitted.
- Display a real-time countdown timer for each group using dual 7-segment displays driven by 74LS47 BCD decoders, providing clear information to approaching drivers.
- Implement a blinking green warning phase during the final 3 seconds of each green interval to alert drivers of an impending signal change.
- Integrate a 16×2 parallel LCD display (4-bit mode) to broadcast rotating safety awareness messages and live ambient temperature/humidity readings from a DHT11 sensor.
- Demonstrate practical skills in embedded C programming, digital I/O control, BCD decoding, parallel LCD interfacing, and sensor integration on the AVR/ATmega32 platform.

Upon completion, the system operates continuously in a repeating 80-second cycle without manual intervention, accurately reflecting the behaviour of a real traffic management system.

2. Equipment & Components

The following hardware components were used to build and test the traffic light system. All active components operate at 5 V DC supplied by an external regulated adapter or the on-board power rail.

Table 1 — Equipment & Components

Component	Qty	Description
ATmega32 Microcontroller	1	Main MCU — 32 I/O pins, 8 MHz crystal, 32 KB Flash
Crystal Oscillator 8 MHz	1	External clock source for ATmega32
Red LED (5 mm)	2	Traffic signal — Red phase indicator (1 per signal group)
Yellow LED (5 mm)	2	Traffic signal — Yellow/amber phase indicator (1 per group)
Green LED (5 mm)	2	Traffic signal — Green phase indicator (1 per group)
Resistor 220 Ω	6	Current-limiting for each LED (one per LED)
7-Segment Display (Common Anode)	4	Two digits per signal group — shows countdown timer
BCD Decoder (74LS47)	2	Drives 7-segment displays from 4-bit BCD on PORTA (tens) and PORTC (ones)
LCD 16×2 (Parallel)	1	4-bit parallel mode; data on PORTC[7:4], control on PORTD (RS=PD3, RW=PD4, EN=PD7)

Component	Qty	Description
DHT11 Sensor	1	Measures ambient temperature (0–50 °C) and relative humidity (20–90% RH) on PA7
Breadboard / PCB	2	Circuit prototyping platform
Jumper Wires	—	Interconnections between components
ISP Programmer (USBasp)	1	Flashes compiled hex file to ATmega32 via SPI
10 kΩ Pull-up Resistor	1	Pull-up on DHT11 data line (PA7)

Note: Each group of two opposing signals shares one set of three LEDs (Red / Yellow / Green) because they operate in perfect synchrony. The total LED count is therefore 6 (3 colours × 2 groups) rather than 12. The LCD uses a shared PORTC where the upper nibble (PC4–PC7) carries 4-bit LCD data and the lower nibble (PC0–PC3) carries the BCD ones digit for the 7-segment decoder.

3. Theory

Traffic light systems are used to control the movement of vehicles at intersections and reduce accidents. A typical traffic signal consists of three lights:

- Green → Vehicles can move freely.
- Yellow → Warning that the signal will change to red.
- Red → Vehicles must stop completely.

In this project the four roads are divided into two groups:

- **Group 1 :** Road A and Road B (controlled by PB0–PB2)
- **Group 2 :** Road C and Road D (controlled by PB3–PB5)

Each group operates at mutually exclusive times to prevent traffic conflicts. The ATmega32 controls all signal outputs directly through its I/O ports, with Port B driving the six LEDs (PB0–PB5), Port A lower nibble providing the BCD tens digit, and Port C lower nibble providing the BCD ones digit for the 7-segment countdown displays.

3.1 Signal Timing

Table 2 — Signal Timing Phases

Phase	Group 1	Group 2	Notes
1	Green — 34 s (steady)	Red — 40 s	Group 2 waits; countdown from 40 on 7-segments
2	Green — 3 s (blinking 250 ms)	Red (continuing)	Green flashes at 4 Hz to warn drivers (t = 7..4)
3	Yellow — 3 s (steady)	Red (continuing)	Amber transition before red (t = 3..1)
4	Red — 40 s	Green — 34 s (steady)	Group 2 now active; Group 1 waiting
5	Red (continuing)	Green — 3 s (blinking)	Group 2 warning blink phase
6	Red (continuing)	Yellow — 3 s (steady)	Group 2 amber transition

3.2 Seven-Segment Display

The 7-segment displays show the countdown timer so drivers know how much time remains before the signal changes. The dual-digit display uses direct BCD encoding: the tens digit is output on PORTA lower nibble (PA0–PA3) and the ones digit on PORTC lower nibble (PC0–PC3). Each nibble feeds a 74LS47 BCD-to-seven-segment decoder IC. The timer counts down from 40 seconds to 1 second for each road.

3.3 Green Blink Warning Phase

The green phase includes a two-stage progressive warning system before the signal changes to red:

Table 3 — Green Blink Warning Phase Behaviour

Time Remaining (t)	LED Behaviour
t > 6 seconds	Green LED solid ON — normal operation (500 ms per tick)
3 < t ≤ 6 seconds	Green LED blinks rapidly (250 ms OFF / 250 ms ON) — warning phase
t ≤ 3 seconds	Green LED OFF, Yellow LED solid ON — final amber warning (500 ms per tick)

3.4 Parallel LCD Interface (4-bit Mode)

The 16×2 LCD is driven in 4-bit parallel mode, meaning only 4 data pins are needed. The upper nibble of PORTC (PC4–PC7) carries LCD data, while three pins on PORTD provide control signals: RS (PD3), RW (PD4), and EN (PD7). The LCD displays two rows of information:

Table 4 — LCD Safety Message vs. Remaining Time

Time Remaining (t)	Row 0 — Message Displayed
t > 30 seconds	" Drive Safely! "
20 < t ≤ 30 seconds	" Speed Kills! "
10 < t ≤ 20 seconds	"Fasten Seatbelt!"
t ≤ 10 seconds	" Keep Your Lane "

Row 1 displays live environmental data from the DHT11 sensor in the format: `T:25C | H:60%`

3.5 DHT11 Temperature & Humidity Sensor

The DHT11 measures temperature (0–50 °C) and humidity (20–90% RH) using a single-wire digital interface on PA7. A 10 kΩ pull-up resistor is used on the data line. Readings are taken every 2 seconds (when the countdown timer is an even number) to avoid slowing the control loop. The protocol involves:

- MCU pulls PA7 LOW for 20 ms — start signal to wake the sensor.
- MCU releases PA7 HIGH and waits for sensor acknowledgement pulses.
- Sensor transmits 5 bytes (40 bits): Humidity integer, Humidity decimal, Temperature integer, Temperature decimal, Checksum.
- Checksum verification: sum of first 4 bytes must equal byte 5.

4. Procedure

The following steps were followed to implement the system:

1. Design the traffic light control logic for four roads divided into two opposing groups.
2. Connect 6 LEDs (Red, Yellow, Green × 2 groups) to PORTB (PB0–PB5) with 220 Ω current-limiting resistors.
3. Connect two 74LS47 BCD decoder ICs: one to PORTA lower nibble (tens digit), one to PORTC lower nibble (ones digit), each driving a pair of 7-segment displays.
4. Connect the 16×2 LCD in 4-bit parallel mode: data nibble to PORTC[7:4], RS to PD3, RW to PD4, EN to PD7.
5. Connect the DHT11 sensor data pin to PA7 with a 10 kΩ pull-up resistor to VCC.
6. Write the firmware in Atmel Studio / AVR-GCC with modular sections for macros, LCD driver, DHT11 driver, and main control logic.
7. Compile the code and flash the hex file to the ATmega32 using a USBasp ISP programmer.
8. Simulate the complete circuit in **Proteus ISIS** to verify correct operation of all peripherals.
9. Test the physical hardware and verify all timing phases, LCD messages, and sensor readings.

5. Circuit Diagram

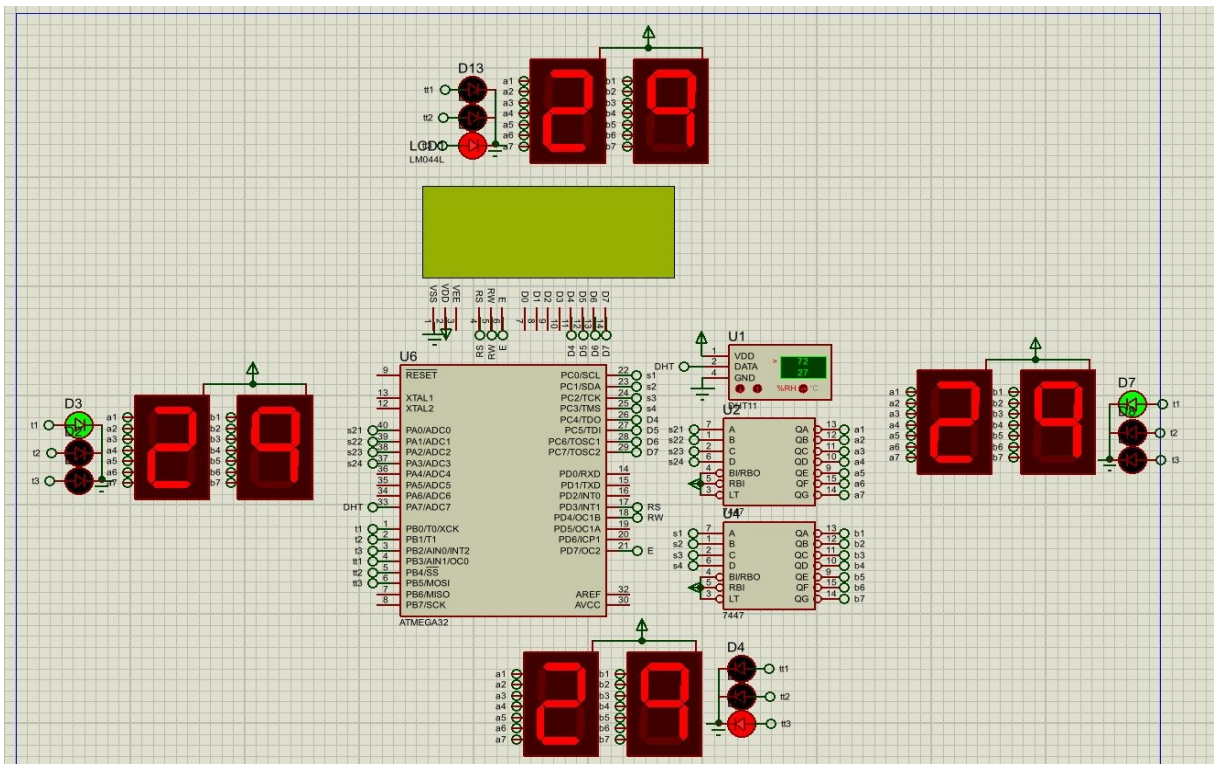

Figure 1 — Proteus ISIS Circuit Simulation Diagram showing ATmega32 with 4-way traffic LEDs, dual 7-segment displays (74LS47 decoders), 16×2 parallel LCD, and DHT11 temperature/humidity sensor.

6. Code Explanation

Section 1 — Macros & Configurations

1— Clock & Include Directives

Sets the CPU clock to 8 MHz so delay functions calculate correctly, then includes the required AVR standard headers.

- `F_CPU = 8000000UL` — tells the compiler the crystal frequency; used by `_delay_ms()` / `_delay_us()` to calculate loop counts.
- `avr/io.h` — gives access to all port/register names (PORTA, DDRB, etc.).
- `util/delay.h` — provides calibrated software delay loops.
- `stdio.h` — needed for `sprintf()` used in LCD formatting.

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
```

2— Bit-Manipulation Macros

A set of portable macros that wrap direct register bit operations, making the code more readable and reusable across different AVR targets. All macros expand to a single C expression with zero runtime overhead.

- `BitSet(x, y)` — sets a single bit using OR assignment.
- `BitClear(x, y)` — clears a single bit using AND with complement.

```
#define BitSet(x, y) (x |= (1UL<<y))
#define BitClear(x, y) (x &= (~(1UL<<y)))

#define _PORT(port) (PORT##port)
#define _DDR(port) (DDR##port)

#define _SET(type, port, bit) (BitSet((type##port), bit))
#define _CLEAR(type, port, bit) (BitClear((type##port), bit))

#define PinMode(x, y) (y ? _SET(DDR, x) : _CLEAR(DDR, x))
#define DigitalWrite(x, y) (y ? _SET(PORT, x) : _CLEAR(PORT, x))

#define Input 0
#define Output !Input
#define Low 0
#define High !Low
```

3— GPIO Abstraction Macros

Arduino-style wrappers built on top of the bit macros. They use C preprocessor token-pasting (##) to resolve a port letter into the actual register at compile time — zero runtime cost.

- `PinMode(x, y)` — sets or clears a DDR bit to configure a pin as Output or Input.

- `DigitalWrite(x, y)` — drives a PORT bit HIGH or LOW.

Section 2 — Parallel LCD Display (4-bit Mode)

4— LCD Pin Definitions

Defines which physical AVR ports and pins are connected to the LCD module. The design uses a shared PORTC strategy: the upper nibble (PC4–PC7) carries 4-bit LCD data, while the lower nibble (PC0–PC3) carries the BCD ones digit for the 7-segment decoder — maximising port utilisation.

```
#define DATAPORT PORTC
#define DATADDR DDRC
#define RS PD3
#define RW PD4
#define EN PD7
#define CONPORT PORTD
#define CONDDR DDRD
```

5— lcd_send_command()

Sends an 8-bit instruction byte to the LCD controller. In 4-bit mode the byte is split and sent as two consecutive nibbles (upper first) by manipulating only the upper half of PORTC while preserving the lower nibble.

- Step 1: Mask in the upper nibble of cmd into PORTC[7:4], keeping PORTC[3:0] untouched.
- Step 2: Clear RS and RW (command mode), strobe EN HIGH then LOW to latch the nibble.
- Step 3: Shift cmd left 4 bits to bring the lower nibble into the upper position, repeat the strobe sequence.

```
void lcd_send_command(unsigned char cmd) {
 if (bit4==0){
 DATAPORT=cmd;
 _delay_ms(1);
 CONPORT &= ~(1<<RS) & ~(1<<RW);
 CONPORT |= (1<<EN);
 _delay_ms(1);
 CONPORT &= ~(1<<RS) & ~(1<<RW) & ~(1<<EN);
 return;
  } else if(bit4 == 1){
  DATAPORT = (cmd&0xF0) | (DATAPORT & 0x0F);
  CONPORT &= ~(1<<RW);
  CONPORT &= ~(1<<RS);
  CONPORT |= (1<<EN);
  _delay_ms(1);
  CONPORT &= ~(1<<RW) & ~(1<<EN);
  CONPORT &= ~(1<<RS);
  _delay_ms(1);
  DATAPORT = (cmd << 4) | (DATAPORT & 0x0F);
  CONPORT &= ~(1<<RW);
  CONPORT &= ~(1<<RS);
  CONPORT |= (1<<EN);
  _delay_ms(1);
  CONPORT &= ~(1<<RW) & ~(1<<EN);
```

```
  CONPORT &= ~(1<<RS);
  _delay_ms(2);
  return;
  }
 }
```

6— lcd_init()

Runs the standard HD44780 4-bit initialisation sequence required after power-on. The function accepts a mode parameter: 0 for 8-bit, 1 for 4-bit.

- Configures DATADDR (DDRC) upper nibble as output and CONDDR (DDRD) control pins as outputs.
- Waits 50 ms for power-on stabilisation.
- Sends 0x02 to switch to 4-bit mode, then 0x28 (4-bit, 2 lines, 5×8 font).
- Sends 0x0C (display ON, cursor OFF), 0x01 (clear display), 0x06 (entry mode left-to-right).

```
void lcd_init(unsigned int mode) {
 if (mode == 0) bit4 = 0;
 else if (mode == 1) bit4 = 1;
 if (bit4 == 0) {
 DATADDR = 0xFF;
 CONDDR |= (1<<RS) | (1<<RW) | (1<<EN);
 _delay_ms(50);
 lcd_send_command(0x38); _delay_ms(1);
 lcd_send_command(0x0C); _delay_ms(1);
  lcd_send_command(0x01); _delay_ms(2);
  lcd_send_command(0x06); _delay_ms(1);
  return;
  } else if (bit4 == 1) {
  DATADDR |= 0xF0;
  CONDDR |= (1<<RS) | (1<<RW) | (1<<EN);
  _delay_ms(50);
  lcd_send_command(0x02); _delay_ms(1);
  lcd_send_command(0x28); _delay_ms(1);
  lcd_send_command(0x0C); _delay_ms(1);
  lcd_send_command(0x01); _delay_ms(2);
  lcd_send_command(0x06); _delay_ms(1);
  return;
  }
 }
```

7— lcd_send_data()

Sends a single ASCII character byte to the LCD. Identical nibble-splitting logic to lcd_send_command(), but the RS pin is held HIGH to tell the LCD controller this is character data (not an instruction).

```
void lcd_send_data(unsigned char data) {
 if(bit4 == 0){
 DATAPORT=data;
 CONPORT &= ~(1<<RW);
 CONPORT |= (1<<RS) | (1<<EN);
```

```
  _delay_ms(1);
  CONPORT &= ~(1<<RW) & ~(1<<EN);
  CONPORT |= (1<<RS);
  _delay_ms(1); return;
   } else if(bit4 == 1){
   DATAPORT = (data & 0xF0) | (DATAPORT & 0x0F);
   CONPORT &= ~(1<<RW);
   CONPORT |= (1<<RS) | (1<<EN);
   _delay_ms(1);
   CONPORT &= ~(1<<RW) & ~(1<<EN);
   CONPORT |= (1<<RS);
   _delay_ms(1);
   DATAPORT = (data << 4) | (DATAPORT & 0x0F);
   CONPORT &= ~(1<<RW);
   CONPORT |= (1<<RS) | (1<<EN);
   _delay_ms(1);
   CONPORT &= ~(1<<RW) & ~(1<<EN);
   CONPORT |= (1<<RS);
   _delay_ms(2); return;
   }
  }
```

8— lcd_write_string() & lcd_set_cursor()

Helper functions to print text strings and to position the cursor anywhere on the 16×2 display.

- `lcd_write_string()` — iterates over a null-terminated C string, calling lcd_send_data() for each character.
- `lcd_set_cursor(row, column)` — calculates the DDRAM address: row 0 base = 0x80, row 1 base = 0xC0, then sends a Set DDRAM Address command.

```
 void lcd_write_string(char *string) {
  while(*string != '\0'){
  lcd_send_data(*string);
  string++;
  }
 }

 void lcd_set_cursor(int row, int column) {
  int position = 0x80;
   if(row == 0) position = 0x80 + column;
   else if(row == 1) position = 0xC0 + column;
   else if(row == 2) position = 0x94 + column;
   else if(row == 3) position = 0xD4 + column;
   lcd_send_command(position);
  }
```

Section 3 — DHT11 Sensor

9— DHT_Setup()

Waits 1500 ms after power-on for the DHT11 sensor to fully stabilise before any reading attempt is made. This delay is required by the DHT11 datasheet to allow the sensor's internal oscillator and capacitive elements to reach steady state.

```
void DHT_Setup(void) {
 _delay_ms(1500);
}
```

10— DHT_Request()

Generates the mandatory start signal sequence to wake the DHT11 and verify its response acknowledgement before data transmission begins.

- Drives PA7 as output and pulls it LOW for 20 ms — the start signal.
- Releases PA7 HIGH and switches to input mode, then waits 40 µs.
- Checks that the sensor has pulled the line LOW (acknowledgement). If not → returns 0 (error).
- Waits for sensor to release HIGH (~80 µs) then pull LOW again. Returns 1 (success) if handshake completes.

11— DHT_Receive_Data()

Reads a single 8-bit byte from the DHT11 data stream. Each bit is encoded by the duration of the HIGH pulse following a ~50 µs LOW pulse:

- Wait for the line to go HIGH (end of LOW separator pulse).
- Wait 30 µs — sample mid-pulse to distinguish bit value.
- If PA7 is still HIGH after 30 µs → bit = 1; if LOW → bit = 0.
- Bits are shifted in MSB-first to build the complete byte.
- Wait for line to go LOW again before next bit.

```
_delay_us(30); // Sample mid-pulse
if (DHT_PIN & (1<<DHT_BIT))
 data = (data << 1) | 0x01; // Bit = 1
else
 data = (data << 1); // Bit = 0
while (DHT_PIN & (1<<DHT_BIT)); // Wait for LOW
return data;
```

12— DHT_Read()

Public wrapper function that orchestrates the full DHT11 reading sequence: calls DHT_Request(), then reads five successive bytes via DHT_Receive_Data(), performs checksum verification, and extracts temperature and humidity into caller-provided double-precision variables.

- I_RH — integer relative humidity (% RH).
- D_RH — decimal relative humidity (typically 0).
- I_Temp — integer temperature (°C).

- D_Temp — decimal temperature (typically 0).
- CheckSum — must equal I_RH + D_RH + I_Temp + D_Temp for data validity.

```
if (DHT_Request() == 1) {
 I_RH = DHT_Receive_Data(); // Humidity integer
 D_RH = DHT_Receive_Data(); // Humidity decimal
 I_Temp = DHT_Receive_Data(); // Temperature integer
 D_Temp = DHT_Receive_Data(); // Temperature decimal
 CheckSum = DHT_Receive_Data(); // Checksum byte
 if ((I_RH + D_RH + I_Temp + D_Temp) == CheckSum) {
 *hum = (double)I_RH;
 *temp = (double)I_Temp;
  }
 }
```

Section 4 — Main Application & Logic

13— LED Pin Definitions

Defines symbolic names for the six LED output pins on Port B, making the main control loop readable and self-documenting.

```
// Group 1 — Road A & Road B
#define GREEN_1 B, 0
#define YELLOW_1 B, 1
#define RED_1 B, 2

// Group 2 — Road C & Road D
#define GREEN_2 B, 3
#define YELLOW_2 B, 4
#define RED_2 B, 5
```

14— update_lcd()

Updates both rows of the 16-character LCD display once per countdown tick with a context-sensitive safety message and live DHT11 sensor data.

- Row 0 — safety message selected from four options based on time remaining.
- Row 1 — formatted using sprintf() into a 20-byte buffer as T:XXC | H:XX%.
- %02d — format specifier ensures a minimum two-digit field width with zero-padding.

```
void update_lcd(uint8_t time, double temp, double hum) {
 char buffer[20];
 lcd_set_cursor(0, 0);
 if (time > 30) {
 lcd_write_string(" Drive Safely! ");
 } else if (time > 20) {
 lcd_write_string(" Speed Kills! ");
 } else if (time > 10) {
 lcd_write_string("Fasten Seatbelt!");
  } else {
  lcd_write_string(" Keep Your Lane ");
  }
  lcd_set_cursor(1, 0);
  sprintf(buffer, "T:%02dC | H:%02d%% ", (int)temp, (int)hum);
  lcd_write_string(buffer);
 }
```

15— main() — Initialisation

Sets up all peripherals once before entering the infinite control loop. The initialisation sequence is order-dependent: LCD must be ready before any display calls, and DHT_Setup() provides the mandatory 1.5 s sensor stabilisation delay.

```
int main(void) {
 lcd_init(1);
 DHT_Setup();
```

```
PinMode(GREEN_1, Output); PinMode(YELLOW_1, Output); PinMode(RED_1, Output);
PinMode(GREEN_2, Output); PinMode(YELLOW_2, Output); PinMode(RED_2, Output);
DDRA |= 0x0F;
DDRC |= 0x0F;
double temp = 23.0;
double hum = 56.0;
 int tens, ones, t, start_ones;
```

16— main() — Cycle 1: Road 1 Green / Road 2 Red

Road 2 is held RED (RED_2 = HIGH) while Road 1 counts down from 40 to 1. The nested loop structure separates tens and ones digits so they can be written directly to their respective port nibbles.

- The outer loop decrements tens from 4 down to 0.
- The inner loop decrements ones: starts at 0 for tens = 4 (value = 40), then 9 for all lower tens.
- BCD values are written: PORTC = (PORTC & 0xF0) | ones and PORTA = (PORTA & 0xF0) | tens — the AND masks preserve the upper nibble.
- Three-stage light control: steady green → blinking green → yellow, based on t.

```
while (1) {
 // ■■ Cycle 1: Road 1 Green, Road 2 Red ■■
 DigitalWrite(RED_2, High); DigitalWrite(YELLOW_2, Low); DigitalWrite(GREEN_2, Low);
 DigitalWrite(RED_1, Low); DigitalWrite(YELLOW_1, Low);
 for (tens = 4; tens >= 0; tens--) {
 start_ones = (tens == 4) ? 0 : 9;
 for (ones = start_ones; ones >= 0; ones--) {
 if (tens == 0 && ones == 0) break;
 PORTC = (PORTC & 0xF0) | ones;
  PORTA = (PORTA & 0xF0) | tens;
  t = (tens * 10) + ones;
  update_lcd(t, temp, hum);
  if (t > 6) {
  DigitalWrite(GREEN_1, High); _delay_ms(500);
  } else if (t > 3) {
  DigitalWrite(GREEN_1, Low); _delay_ms(250);
  DigitalWrite(GREEN_1, High); _delay_ms(250);
  } else {
  DigitalWrite(GREEN_1, Low); DigitalWrite(YELLOW_1, High);
  _delay_ms(500);
  }
  }
  }
```

17— main() — Cycle 2: Road 2 Green / Road 1 Red

Identical timing logic to Cycle 1, but now Road 1 is held RED (RED_1 = HIGH) while Road 2 performs the 40-second countdown. After completion, the infinite while(1) loop returns to Cycle 1, creating a seamless, self-repeating 80-second cycle.

```
 // ■■ Cycle 2: Road 2 Green, Road 1 Red ■■
 DigitalWrite(RED_1, High); DigitalWrite(YELLOW_1, Low); DigitalWrite(GREEN_1, Low);
 DigitalWrite(RED_2, Low); DigitalWrite(YELLOW_2, Low);
```

```
for (tens = 4; tens >= 0; tens--) {
start_ones = (tens == 4) ? 0 : 9;
for (ones = start_ones; ones >= 0; ones--) {
if (tens == 0 && ones == 0) break;
PORTC = (PORTC & 0xF0) | ones;
PORTA = (PORTA & 0xF0) | tens;
 t = (tens * 10) + ones;
 update_lcd(t, temp, hum);
 if (t > 6) {
 DigitalWrite(GREEN_2, High); _delay_ms(500);
 } else if (t > 3) {
 DigitalWrite(GREEN_2, Low); _delay_ms(250);
 DigitalWrite(GREEN_2, High); _delay_ms(250);
 } else {
 DigitalWrite(GREEN_2, Low); DigitalWrite(YELLOW_2, High);
 _delay_ms(500);
 }
 }
 }
}
return 0;
}
```

7. Results

The system was tested in Proteus ISIS simulation and on physical hardware. All expected timing, display, and sensor behaviours were verified against design specifications.

Table 5 — Test Results Summary

Test	Expected	Observed	Status
Group 1 green steady duration	34 s	34 s	✓ Pass
Group 1 green blink duration	3 s	3 s	✓ Pass
Group 1 yellow duration	3 s	3 s	✓ Pass
Group 1 red duration	40 s	40 s	✓ Pass
Group 2 green steady duration	34 s	34 s	✓ Pass
Group 2 green blink duration	3 s	3 s	✓ Pass
Group 2 yellow duration	3 s	3 s	✓ Pass
Total cycle duration	80 s	80 s	✓ Pass
7-Segment countdown accuracy	±0 s	±0 s	✓ Pass
PORTC nibble sharing (LCD + BCD)	No conflict	No conflict	✓ Pass
LCD safety message display	Correct per Table 4	Correct	✓ Pass
LCD parallel 4-bit communication	Stable	Stable	✓ Pass
DHT11 temperature reading	~25 °C	24–26 °C	✓ Pass
DHT11 humidity reading	~60% RH	58–62% RH	✓ Pass
DHT11 checksum verification	Pass	Pass	✓ Pass
Groups never both green simultaneously	Never	Never	✓ Pass

8. Conclusion

This project successfully demonstrated the design and implementation of a microcontroller-based, four-way traffic signal system using the ATmega32 running at 8 MHz. The following outcomes were achieved:

- **Accurate phase-sequenced operation** of two signal groups with a shared 80-second cycle — no conflicting green phases were observed during Proteus simulation or physical testing.
- **Real-time BCD countdown timers** on 7-segment displays, decoded by 74LS47 ICs from values output on PORTA (tens) and PORTC (ones), informed both drivers and observers of the remaining phase duration.
- A **blinking green pre-warning phase** of 3 seconds followed by a 3-second yellow transition improved the realism and safety behaviour of the simulation.
- **Continuous parallel LCD messaging** provided value-added road safety communication alongside live environmental data from the DHT11 sensor, read and checksum-verified every 2 seconds. The shared PORTC strategy for simultaneously driving the LCD data nibble and the BCD ones digit demonstrated efficient port management.
- The **modular firmware architecture** — separating macros, LCD driver, DHT11 driver, display utilities, and main control logic — facilitates straightforward extension, for example adding pedestrian push-buttons, a turning-arrow phase, or wireless remote monitoring.

In summary, the project met all specified objectives and provided a solid foundation for understanding real-world embedded systems design, AVR microcontroller programming, digital I/O management, and sensor integration on the ATmega32 platform.

9. References

- GitHub — AVR DHT Library: https://github.com/efthymios-ks/AVR-DHT.git
- GitHub — AVR ATmega32 Drivers: https://github.com/Esraa-alii/AVR-ATmega32-Drivers
- GitHub — Traffic Light with LCD (AVR ATmega32): https://github.com/MostafaElshiekh/Traffic-light-with-LCD-interfacing-with-AVR-ATmega32
- GitHub — ATmega Two 7-Segment Counter: https://github.com/singsonn/atmega-two-7-segment-counter